

Báo cáo training Data Engineer

Họ và tên	:	Hoàng Quang Huy
Ngày tháng	:	04/08/2026
Người hướng dẫn	:	

Mục lục

1. SQL.....	3
1.1 DDL, DML, DCL.....	3
1.2 Filtering, Aggregation, Join.....	5
1.3 Subqueries & CTE.....	10
1.4 Windows Function.....	11
1.5 View.....	11
2. OLTP và OLAP.....	12
2.1 OLTP.....	12
2.2 OLAP.....	13
2.3 Phân biệt OLTP và OLAP.....	14
3. ACID và CAP Theorem.....	15
3.1 ACID.....	15
3.2 CAP Theorem.....	17
4. Các cơ chế của hệ quản trị cơ sở dữ liệu.....	18
4.1 Index.....	18
4.2 Transaction.....	18
4.3 Lock.....	19
4.4 Partition.....	20
4.5 Sharding.....	22
4.6 Isolation.....	23
4.7 Connection Pooling.....	25
5. Nhận xét, lưu ý trong quá trình truy vấn.....	26

1. SQL

1.1 DDL, DML, DCL

- **DDL** là một thành phần trong SQL, chứa các câu lệnh tạo/xoá các đối tượng cơ sở dữ liệu như bảng, index và thiết lập các ràng buộc quan hệ. Các câu lệnh DDL quan trọng:

- **CREATE**

- + Tạo một cơ sở dữ liệu mới

CREATE DATABASE <database>

Ví dụ: Cơ sở dữ liệu “myTV”

CREATE DATABASE myTV;

- + Tạo một bảng mới

CREATE TABLE <bảng>

Ví dụ: Bảng “Customer”

CREATE TABLE Customer(

customer_id INT PRIMARY KEY,

customer_name VARCHAR(50),

age INT

);

- + Tạo một chỉ mục

CREATE INDEX <tên index>

ON <bảng> (<thuộc tính>)

Ví dụ: Cột “customer_name” trong bảng “Customer”

CREATE INDEX idx_customer_name

ON Customer (customer_name);

- **ALTER**

- + Thêm cột mới cho bảng

Ví dụ: Thêm cột “phone_number” cho bảng “Customer”

ALTER TABLE Customer

ADD COLUMN phone_number VARCHAR(20);

+ Xoá cột của bảng

Ví dụ: Xoá cột “age” của bảng “Customer”

ALTER TABLE Customer

DROP COLUMN age;

+ Thay đổi kiểu dữ liệu một cột của bảng

Ví dụ: Thay đổi kiểu dữ liệu cột “phone_number” cho bảng “Customer”

ALTER TABLE Customer

CHANGE COLUMN phone_number VARCHAR(10);

- **DROP** (tương tự CREATE)

+ Xóa cơ sở dữ liệu, xóa bảng, xóa chỉ mục

Ví dụ: Xóa bảng “Customer”

DROP TABLE Customer;

- **DML** là một phần quan trọng của SQL, chứa các câu lệnh cho phép thao tác với dữ liệu trong các bảng cơ sở dữ liệu. Các câu lệnh quan trọng trong DML:

- **SELECT**

SELECT [DISTINCT] <thuộc tính>

FROM <bảng>

- **UPDATE**

UPDATE <bảng>

SET <thuộc tính> = <giá trị>

[WHERE <biểu thức điều kiện>]

- **DELETE**

DELETE FROM <bảng>

WHERE <biểu thức điều kiện>;

- **INSERT INTO**

INSERT INTO <bảng> (<Danh sách các thuộc tính>)

VALUES (<danh sách các giá trị tương ứng>);

- **DCL** là một phần quan trọng của SQL và được sử dụng để kiểm soát quyền truy cập và phân quyền cho người dùng và vai trò. Một số câu lệnh quan trọng trong DCL:

- **GRANT**

GRANT <quyền>

ON <bảng, cơ sở dữ liệu>

TO <đối tượng>

- **REVOKE**

REVOKE <quyền>

ON <bảng, cơ sở dữ liệu>

TO <đối tượng>

- **CREATE ROLE**

CREATE ROLE <vai trò>

- **ALTER ROLE**

ALTER ROLE <vai trò> ADD <quyền>

- **DROP ROLE**

DROP ROLE <đối tượng>

1.2 Filtering, Aggregation, Join

Filtering là quá trình lựa chọn các bản ghi từ một hoặc nhiều bảng dựa trên các điều kiện xác định. Mục đích của Filtering là loại bỏ các dữ liệu không cần thiết và chỉ giữ lại các bản ghi phục vụ cho yêu cầu truy vấn.

Các câu lệnh thường dùng:

- **WHERE**

WHERE thường kết hợp với các câu lệnh trong DML (SELECT, INSERT INTO, DELETE, UPDATE) với vai trò là lọc các bản ghi thoả mãn điều kiện.

Ví dụ:

SELECT <thuộc tính>

FROM <bảng>

WHERE <biểu thức điều kiện>

- **HAVING**

HAVING là câu lệnh dùng để lọc các nhóm dữ liệu sau khi đã thực hiện phép nhóm (GROUP BY). Mệnh đề này thường được sử dụng để lọc kết quả của các hàm tổng hợp như COUNT(), SUM(), AVG(), MAX() và MIN().

Ví dụ: Tính tổng doanh thu của từng cửa hàng và chỉ hiển thị các cửa hàng có doanh thu lớn hơn 100 triệu đồng.

SELECT StoreID, SUM(Revenue) AS TotalRevenue

FROM Sales

GROUP BY StoreID

HAVING SUM(Revenue) > 100;

Aggregation là quá trình tổng hợp dữ liệu từ nhiều bản ghi để tạo ra một giá trị thống kê duy nhất. Các hàm này giúp thực hiện các phép tính trên một tập dữ liệu, chẳng hạn như đếm số lượng bản ghi, tính tổng, tính trung bình hoặc xác định giá trị lớn nhất và nhỏ nhất.

Các câu lệnh thường dùng:

- **COUNT**

COUNT là câu lệnh để đếm số lượng bản ghi

Ví dụ: Đếm số lượng nhân viên trong bảng Employees

SELECT COUNT() AS TotalEmployees*

FROM Employees;

- **SUM**

SUM được sử dụng để tính tổng giá trị một cột kiểu số.

Ví dụ: Tính tổng doanh thu của bảng Sales.

SELECT SUM(Revenue) AS TotalRevenue

FROM Sales;

- **AVG**

AVG được sử dụng để tính giá trị trung bình của một cột kiểu số.

Ví dụ: Tính mức lương trung bình của nhân viên trong bảng Employees.

```
SELECT AVG(Salary) AS AverageSalary
```

```
FROM Employees;
```

- **MAX**

MAX được sử dụng để tìm giá trị lớn nhất trong một cột.

Ví dụ: Tìm mức lương cao nhất trong bảng Employees.

```
SELECT MAX(Salary) AS HighestSalary
```

```
FROM Employees;
```

- **MIN**

MIN được sử dụng để tìm giá trị nhỏ nhất trong một cột.

Ví dụ: Tìm mức lương thấp nhất trong bảng Employees.

```
SELECT MIN(Salary) AS LowestSalary
```

```
FROM Employees;
```

- **GROUP BY**

GROUP BY được sử dụng để nhóm các bản ghi có cùng giá trị của một hoặc nhiều cột, từ đó áp dụng các hàm tổng hợp cho từng nhóm dữ liệu.

Ví dụ: Đếm số lượng nhân viên theo từng phòng ban.

```
SELECT Department, COUNT(*) AS TotalEmployees
```

```
FROM Employees
```

```
GROUP BY Department;
```

Join là thao tác kết hợp dữ liệu từ hai hoặc nhiều bảng dựa trên mối quan hệ giữa các cột chung của các bảng. Join thường được sử dụng khi dữ liệu được lưu trữ ở nhiều bảng khác nhau nhưng cần truy vấn thành một tập kết quả duy nhất.

Các câu lệnh thường dùng:

- **INNER JOIN**

INNER JOIN được sử dụng để kết hợp dữ liệu từ hai bảng và chỉ trả về các bản ghi có giá trị khớp ở cả hai bảng.

Ví dụ: Hiển thị tên khách hàng và số tiền đơn hàng.

```
SELECT Customers.CustomerName,  
       Orders.Amount  
FROM Customers  
INNER JOIN Orders  
ON Customers.CustomerID = Orders.CustomerID;
```

- **LEFT JOIN**

LEFT JOIN được sử dụng để trả về tất cả các bản ghi của bảng bên trái và các bản ghi khớp ở bảng bên phải. Nếu không có bản ghi tương ứng ở bảng bên phải thì các cột của bảng này sẽ có giá trị NULL.

Ví dụ: Hiển thị tất cả khách hàng, kể cả những khách hàng chưa có đơn hàng.

```
SELECT Customers.CustomerName,  
       Orders.Amount  
FROM Customers  
LEFT JOIN Orders  
ON Customers.CustomerID = Orders.CustomerID;
```

- **RIGHT JOIN**

RIGHT JOIN được sử dụng để trả về tất cả các bản ghi của bảng bên phải và các bản ghi khớp ở bảng bên trái. Nếu không có bản ghi tương ứng ở bảng bên trái thì các cột của bảng này sẽ có giá trị NULL.

Ví dụ: Hiển thị tất cả đơn hàng, kể cả những đơn hàng chưa có thông tin khách hàng.

```
SELECT Customers.CustomerName,  
       Orders.Amount  
FROM Customers
```


*RIGHT JOIN Orders**ON Customers.CustomerID = Orders.CustomerID;*

- **FULL OUTER JOIN**

FULL OUTER JOIN được sử dụng để trả về tất cả các bản ghi của cả hai bảng. Nếu một bản ghi không có dữ liệu tương ứng ở bảng còn lại thì các cột của bảng đó sẽ có giá trị NULL.

Ví dụ: Hiển thị tất cả khách hàng và tất cả đơn hàng.

*SELECT Customers.CustomerName,**Orders.Amount**FROM Customers**FULL OUTER JOIN Orders**ON Customers.CustomerID = Orders.CustomerID;*

- **CROSS JOIN**

CROSS JOIN được sử dụng để kết hợp mọi bản ghi của bảng thứ nhất với mọi bản ghi của bảng thứ hai. Nếu bảng thứ nhất có m bản ghi và bảng thứ hai có n bản ghi thì kết quả sẽ có $m \times n$ bản ghi.

Ví dụ: Kết hợp tất cả sản phẩm với tất cả màu sắc.

*SELECT Products.ProductName,**Colors.ColorName**FROM Products**CROSS JOIN Colors;*

- **SELF JOIN**

SELF JOIN được sử dụng để kết nối một bảng với chính nó. Phép nối này thường được áp dụng khi các bản ghi trong cùng một bảng có mối quan hệ với nhau.

Ví dụ: Hiển thị tên nhân viên và tên người quản lý của họ.

*SELECT e.EmployeeName,**m.EmployeeName AS Manager*

```
FROM Employees e
LEFT JOIN Employees m
ON e.ManagerID = m.EmployeeID;
```

1.3 Subqueries & CTE

Subquery là một câu lệnh SQL được đặt bên trong một câu lệnh SQL khác. Subquery được sử dụng để tạo ra một kết quả trung gian, sau đó kết quả này được truy vấn chính sử dụng để tiếp tục xử lý.

Các toán tử thường dùng với Subquery:

- **ANY**

ANY được sử dụng để so sánh một giá trị với ít nhất một giá trị trong kết quả của Subquery.

Ví dụ: Tìm nhân viên có mức lương lớn hơn ít nhất một mức lương của phòng HR.

```
SELECT Name, Salary
FROM Employees
WHERE Salary > ANY
(
    SELECT Salary
    FROM Employees
    WHERE Department = 'HR'
);
```

- **ALL**

ALL được sử dụng để so sánh một giá trị với tất cả các giá trị trong kết quả của Subquery.

Ví dụ: Tìm nhân viên có mức lương lớn hơn tất cả mức lương của phòng HR.

```
SELECT Name, Salary
FROM Employees
WHERE Salary > ALL
(
```

```
SELECT Salary  
FROM Employees  
WHERE Department = 'HR'  
);
```

CTE (Common Table Expression) là một tập kết quả tạm thời được đặt tên, được định nghĩa bằng từ khóa **WITH** và chỉ tồn tại trong phạm vi của một câu lệnh SQL. CTE giúp chia một truy vấn phức tạp thành nhiều bước xử lý, từ đó làm cho câu lệnh SQL dễ đọc, dễ hiểu và dễ bảo trì hơn.

Ví dụ: Tính tổng doanh thu của từng khách hàng và chỉ hiển thị những khách hàng có doanh thu lớn hơn 100 triệu đồng.

```
WITH CustomerRevenue AS  
(  
    SELECT CustomerID,  
           SUM(Revenue) AS TotalRevenue  
    FROM Sales  
    GROUP BY CustomerID  
)  
SELECT CustomerID,  
       TotalRevenue  
FROM CustomerRevenue  
WHERE TotalRevenue > 100;
```

1.4 Windows Function

Window Function là các hàm trong SQL được sử dụng để thực hiện các phép tính trên một tập hợp các hàng có liên quan đến hàng hiện tại. Khác với các hàm tổng hợp (SUM(), COUNT(), AVG(),...), Window Function không làm giảm số lượng bản ghi mà vẫn giữ nguyên từng bản ghi trong kết quả truy vấn.

1.5 View

View là một bảng ảo (Virtual Table) được tạo từ kết quả của một câu lệnh SELECT. View không lưu trữ dữ liệu riêng mà chỉ lưu trữ câu lệnh truy vấn. Khi truy vấn View, hệ quản trị cơ sở dữ liệu sẽ thực hiện câu lệnh SELECT tương ứng để lấy dữ liệu từ các bảng gốc.

Các câu lệnh thường dùng:

- **CREATE VIEW**

CREATE VIEW <tên view> AS

<câu truy vấn>;

Ví dụ: Tạo một View chứa thông tin các nhân viên thuộc phòng IT.

CREATE VIEW IT_Employees AS

SELECT EmployeeID,

EmployeeName,

Salary

FROM Employees

WHERE Department = 'IT';

Sau khi tạo View, có thể truy vấn như một bảng thông thường.

SELECT *

FROM IT_Employees;

- **DROP VIEW**

DROP VIEW <tên view>;

- **CREATE OR REPLACE VIEW** (tạo mới hoặc thay thế view đã tồn tại)

CREATE OR REPLACE <tên view> AS

<câu truy vấn mới>;

2. OLTP và OLAP

2.1 OLTP

OLTP (Online Transaction Processing) là hệ thống xử lý giao dịch trực tuyến, được thiết kế để quản lý và xử lý các giao dịch phát sinh hằng ngày của người dùng một cách nhanh chóng và chính xác. Các giao dịch này thường bao gồm thêm, sửa, xóa hoặc truy vấn dữ liệu trong cơ sở dữ liệu.

OLTP được sử dụng trong các hệ thống yêu cầu nhiều người dùng truy cập đồng thời và xử lý số lượng lớn các giao dịch trong thời gian thực.

Đặc điểm chính:

- Xử lý số lượng lớn các giao dịch trực tuyến.
- Thời gian phản hồi nhanh, thường chỉ trong vài mili giây đến vài giây.
- Mỗi giao dịch chỉ tác động đến một lượng nhỏ dữ liệu.
- Hỗ trợ nhiều người dùng truy cập và thực hiện giao dịch đồng thời.
- Dữ liệu được cập nhật thường xuyên.
- Đảm bảo tính toàn vẹn của dữ liệu thông qua các thuộc tính ACID (Atomicity, Consistency, Isolation, Durability).

Ví dụ: Hệ thống ngân hàng, hệ thống thương mại điện tử, hệ thống quản lý bán hàng, hệ thống quản lý sinh viên,...

2.2 OLAP

OLAP (Online Analytical Processing) là hệ thống xử lý phân tích trực tuyến, được thiết kế để hỗ trợ việc phân tích dữ liệu, thống kê và ra quyết định. OLAP cho phép người dùng thực hiện các truy vấn phức tạp trên khối lượng lớn dữ liệu lịch sử nhằm phát hiện xu hướng, đánh giá hiệu quả hoạt động và hỗ trợ xây dựng chiến lược kinh doanh.

OLAP thường được sử dụng trong các hệ thống kho dữ liệu (Data Warehouse) và các công cụ Business Intelligence (BI).

Đặc điểm chính:

- Xử lý các truy vấn phân tích và thống kê trên khối lượng dữ liệu lớn.
- Dữ liệu chủ yếu là dữ liệu lịch sử, được tổng hợp từ nhiều nguồn.
- Các truy vấn thường sử dụng các phép tổng hợp như SUM, COUNT, AVG, MAX, MIN.
- Thời gian thực hiện truy vấn thường lâu hơn so với OLTP.
- Dữ liệu ít được cập nhật trực tiếp mà chủ yếu phục vụ cho việc đọc và phân tích.
- Hỗ trợ ra quyết định thông qua các báo cáo và dashboard.

Ví dụ: Hệ thống phân tích doanh thu, hệ thống phân tích xu hướng mua sắm của khách hàng,...

2.3 Phân biệt OLTP và OLAP

Tóm lại, OLTP là nơi phát sinh các giao dịch hằng ngày, tạo hoặc cập nhật dữ liệu trong cơ sở dữ liệu. Dữ liệu từ hệ thống OLTP sẽ được đưa sang kho dữ liệu thông qua quy trình ETL (Extract – Transform – Load) hoặc ELT (Extract – Load – Transform). Sau khi có dữ liệu trong kho dữ liệu, OLAP sẽ thực hiện các truy vấn phân tích.

Người dùng → OLTP → Kho dữ liệu → OLAP

So sánh OLTP và OLAP

Tiêu chí	OLTP	OLAP
Mục đích	Hướng tới xử lý các lượng lớn dữ liệu giao dịch trong thời gian thực.	Chủ yếu để phân tích dữ liệu chi tiết qua nhiều chiều để hỗ trợ quyết định và giải quyết vấn đề
Ứng dụng	ERP, CRM, hệ thống bán hàng, ngân hàng, thương mại điện tử, quản lý bệnh viện, ứng dụng web.	Kho dữ liệu (Data Warehouse), Business Intelligence (BI), hệ hỗ trợ ra quyết định (DSS), dashboard, báo cáo phân tích.
Người dùng	Khách hàng, nhân viên nghiệp vụ.	Nhà quản lý, lãnh đạo, chuyên viên phân tích dữ liệu.
Phạm vi xử lý	Xử lý từng giao dịch riêng lẻ theo thời gian thực.	Phân tích dữ liệu lịch sử trên quy mô lớn theo nhiều chiều.
Cập nhật dữ liệu	Liên tục, ngay khi phát sinh giao dịch.	Theo chu kỳ (hàng giờ, hằng ngày, hằng tuần hoặc định kỳ).
Cấu trúc dữ liệu	Mô hình quan hệ (Entity–Relationship).	Mô hình đa chiều (Multidimensional Data Model) hoặc mô hình quan hệ.
Schema	Chuẩn hóa (Normalized Schema).	Star Schema hoặc Snowflake Schema (thường phi chuẩn hóa một phần).

Tốc độ	Mili giây.	Giây đến giờ
Nhấn mạnh	Cập nhật dữ liệu.	Truy vấn dữ liệu.
Số lượng người dùng	Hàng nghìn đến hàng triệu.	Hàng chục đến hàng trăm.
Dung lượng dữ liệu	MB đến GB hoặc vài TB tùy hệ thống.	GB đến nhiều TB hoặc PB.
Đo lường hiệu năng	Số lượng giao dịch/giây (TPS), thời gian phản hồi giao dịch.	Thời gian thực hiện truy vấn và khả năng xử lý dữ liệu lớn.

3. ACID và CAP Theorem

3.1 ACID

ACID (Atomicity - Consistency - Isolation - Durability) là tập hợp bốn thuộc tính quan trọng của một giao dịch (Transaction) trong hệ quản trị cơ sở dữ liệu, giúp đảm bảo tính chính xác, nhất quán và tin cậy của dữ liệu khi thực hiện các thao tác như thêm, sửa hoặc xóa dữ liệu.

ACID là nền tảng của các hệ thống OLTP, nơi các giao dịch phải được xử lý chính xác ngay cả khi có nhiều người dùng truy cập đồng thời hoặc xảy ra sự cố.

Atomicity đảm bảo rằng một giao dịch được thực hiện hoàn toàn hoặc không thực hiện gì cả. Giao dịch được coi là 1 đơn vị không thể phân chia, nếu một thao tác trong giao dịch thất bại, toàn bộ giao dịch sẽ bị hủy (Rollback), đưa cơ sở dữ liệu trở về trạng thái trước khi giao dịch bắt đầu.

Ví dụ:

Một giao dịch chuyển 1.000.000 đồng từ tài khoản A sang tài khoản B gồm hai bước:

1. Trừ 1.000.000 đồng từ tài khoản A.

2. Cộng 1.000.000 đồng vào tài khoản B.

Nếu sau khi trừ tiền ở tài khoản A mà hệ thống gặp lỗi trước khi cộng tiền cho tài khoản B thì toàn bộ giao dịch sẽ bị hủy và số dư của tài khoản A sẽ được khôi phục như ban đầu.

Consistency đảm bảo rằng sau khi giao dịch hoàn thành, cơ sở dữ liệu luôn chuyển từ một trạng thái hợp lệ sang một trạng thái hợp lệ khác và luôn tuân thủ các ràng buộc dữ liệu đã được định nghĩa.

Ví dụ:

Giả sử quy định

- *Số dư tài khoản không được nhỏ hơn 0.*

Nếu một giao dịch rút tiền làm số dư trở thành âm thì giao dịch sẽ bị từ chối và dữ liệu vẫn giữ nguyên trạng thái hợp lệ.

Isolation đảm bảo rằng các giao dịch đang thực hiện đồng thời không ảnh hưởng lẫn nhau. Mỗi giao dịch được xử lý như thể nó là giao dịch duy nhất đang chạy trên hệ thống.

Ví dụ:

Giả sử cùng lúc

- *Giao dịch A chuyển 500.000 đồng.*
- *Giao dịch B rút 1.000.000 đồng.*

Nếu không có Isolation, hai giao dịch có thể đọc cùng một số dư ban đầu và dẫn đến kết quả sai.

Durability đảm bảo rằng sau khi một giao dịch đã được xác nhận thành công (Commit), dữ liệu sẽ được lưu trữ vĩnh viễn và không bị mất ngay cả khi hệ thống gặp sự cố như mất điện hoặc khởi động lại.

Ví dụ:

Một khách hàng hoàn tất thanh toán đơn hàng.

Ngay sau khi giao dịch được Commit, hệ thống mất điện.

Khi hệ thống hoạt động trở lại, thông tin đơn hàng và thanh toán vẫn được lưu trong cơ sở dữ liệu.

3.2 CAP Theorem

CAP Theorem (Định lý CAP), do nhà khoa học máy tính Eric Brewer đề xuất năm 2000 và được chứng minh bởi Gilbert và Lynch năm 2002, phát biểu rằng:

Một hệ cơ sở dữ liệu phân tán (Distributed Database) không thể đồng thời đảm bảo cả ba thuộc tính: Consistency (C), Availability (A) và Partition Tolerance (P). Khi xảy ra phân vùng mạng (Network Partition), hệ thống chỉ có thể lựa chọn tối đa hai trong ba thuộc tính.

CAP Theorem là một trong những nguyên lý quan trọng trong thiết kế các hệ cơ sở dữ liệu phân tán và các hệ thống NoSQL.

- **Consistency** đảm bảo rằng tất cả các nút (Node) trong hệ thống luôn nhìn thấy cùng một dữ liệu tại cùng một thời điểm. Để điều này xảy ra, bất cứ khi nào dữ liệu được ghi vào một node, nó phải được chuyển tiếp hoặc sao chép ngay lập tức tới tất cả các node khác trong hệ thống trước khi việc ghi được coi là "thành công".
- **Availability** đảm bảo rằng mọi yêu cầu gửi đến hệ thống đều nhận được phản hồi, kể cả khi một số máy chủ gặp sự cố. Hay nói cách khác — đối với bất kỳ yêu cầu nào, tất cả các node đang hoạt động trong hệ thống phân tán phải trả về phản hồi hợp lệ.
- **Partition Tolerance** là khả năng hệ thống vẫn tiếp tục hoạt động khi xảy ra lỗi mạng khiến các máy chủ không thể liên lạc với nhau.

Ngày nay, các cơ sở dữ liệu NoSQL được phân loại dựa trên các đặc điểm trong định lý CAP mà chúng hỗ trợ:

1. Cơ sở dữ liệu CP: Một cơ sở dữ liệu CP có tính nhất quán và dung sai phân vùng, tức hi sinh tính khả dụng. Khi tình trạng phân vùng xảy ra giữa hai node bất kỳ, hệ thống phải shutdown node gặp tình trạng không nhất quán (tức là làm cho nó không khả dụng) cho đến khi việc phân vùng được giải quyết.
2. Cơ sở dữ liệu AP: Cơ sở dữ liệu AP cung cấp tính khả dụng và khả năng chịu phân vùng, tức hi sinh tính nhất quán. Khi tình trạng phân vùng xảy ra, tất cả các node vẫn sẽ khả dụng nhưng sẽ có những node chứa phiên bản dữ liệu cũ hơn những node khác. (Khi tình trạng phân vùng được giải quyết, cơ sở dữ liệu AP thường đồng bộ lại các node để đồng bộ lại tất cả các điểm không nhất quán trong hệ thống.)
3. Cơ sở dữ liệu CA: Cơ sở dữ liệu CA cung cấp tính nhất quán và tính khả dụng trên tất cả các node. Tuy nhiên, nó không thể thực hiện được điều này nếu xảy ra tình

trạng phân vùng giữa hai node bất kỳ trong hệ thống và do đó không có khả năng chịu lỗi (partition tolerance).

4. Các cơ chế của hệ quản trị cơ sở dữ liệu

4.1 Index

- **Khái niệm**

Index (Chỉ mục) là một cấu trúc dữ liệu đặc biệt được hệ quản trị cơ sở dữ liệu (DBMS) tạo ra trên một hoặc nhiều cột của bảng nhằm tăng tốc độ truy vấn dữ liệu.

Thay vì phải quét toàn bộ bảng để tìm kiếm bản ghi phù hợp, DBMS sẽ tra cứu thông qua Index để xác định nhanh vị trí của dữ liệu cần truy xuất.

- **Cơ chế hoạt động**

Giả sử bảng Employees có 1000 bản ghi, nếu không sử dụng index, trong trường hợp xấu nhất DBMS sẽ phải đọc 1000 bản ghi để tìm được các bản ghi thỏa mãn. Khi này, độ phức tạp xấp xỉ $O(n)$.

Khi tạo một Index, DBMS sẽ xây dựng một cấu trúc dữ liệu riêng biệt dựa trên giá trị của cột được lập chỉ mục. Cấu trúc này không lưu toàn bộ dữ liệu của bảng mà chỉ lưu **khóa tìm kiếm (Search Key)** và **con trỏ (Pointer)** đến vị trí của bản ghi trong bảng dữ liệu.

- **Cơ chế tổ chức Index**

Nếu Index chỉ được lưu dưới dạng một danh sách thông thường thì việc tìm kiếm vẫn phải duyệt lần lượt từng phần tử, do đó hiệu quả không cao. Vì vậy, hầu hết các hệ quản trị cơ sở dữ liệu quan hệ hiện nay như MySQL, PostgreSQL, SQL Server và Oracle đều sử dụng cấu trúc dữ liệu **B+ Tree** để tổ chức Index.

B+ Tree là một cây cân bằng, trong đó các khóa được sắp xếp theo thứ tự và được phân bố trên nhiều nút (Node). Khi tìm kiếm một giá trị, DBMS chỉ cần so sánh khóa tại các nút trên đường đi từ nút gốc (Root Node) đến nút lá (Leaf Node) thay vì duyệt toàn bộ dữ liệu.

4.2 Transaction

- **Khái niệm**

Transaction (giao dịch) là một tập hợp gồm một hoặc nhiều câu lệnh SQL được thực hiện như một đơn vị công việc (Unit of Work). Một Transaction chỉ được coi

là hoàn thành khi tất cả các thao tác trong giao dịch được thực hiện thành công, gọi là commit; nếu có bất kỳ thao tác nào xảy ra lỗi, toàn bộ giao dịch sẽ bị hủy và cơ sở dữ liệu được khôi phục về trạng thái trước khi giao dịch bắt đầu, gọi là rollback.

- **Các thuộc tính**

Một Transaction phải tuân thủ theo mô hình **ACID**:

- + Atomicity: Transaction được thực hiện toàn bộ hoặc không thực hiện gì cả.
- + Consistency: Transaction đưa cơ sở dữ liệu từ một trạng thái hợp lệ sang một trạng thái hợp lệ khác.
- + Isolation: Các Transaction đồng thời không ảnh hưởng lẫn nhau.
- + Durability: Sau khi COMMIT, dữ liệu được lưu trữ bền vững ngay cả khi hệ thống gặp sự cố.

4.3 Lock

- **Khái niệm**

Lock là cơ chế đồng bộ hóa (synchronization mechanism) cho phép DBMS kiểm soát quyền truy cập đồng thời của nhiều Transaction lên cùng một dữ liệu nhằm đảm bảo tính nhất quán và toàn vẹn của cơ sở dữ liệu.

- **Cơ chế hoạt động**

Khi một Transaction muốn truy cập hoặc cập nhật dữ liệu, DBMS sẽ tự động kiểm tra trạng thái khóa của dữ liệu đó.

- + Nếu dữ liệu chưa bị khóa hoặc loại khóa tương thích với Transaction hiện tại, DBMS sẽ cấp Lock và cho phép Transaction tiếp tục thực hiện.
- + Nếu dữ liệu đang bị khóa bởi một Transaction khác và hai loại khóa không tương thích, Transaction mới sẽ chuyển sang trạng thái **Lock Wait** và phải chờ cho đến khi Transaction trước giải phóng khóa bằng lệnh COMMIT hoặc ROLLBACK.

- **Các loại khoá**

- + Shared Lock (S Lock)

Shared Lock được cấp khi Transaction chỉ đọc dữ liệu.

Nhiều Transaction có thể đồng thời giữ Shared Lock trên cùng một dữ liệu vì việc đọc không làm thay đổi nội dung dữ liệu.

+ Exclusive Lock (X Lock)

Exclusive Lock được cấp khi Transaction thực hiện thay đổi dữ liệu thông qua các câu lệnh INSERT, UPDATE hoặc DELETE.

Trong thời gian Transaction giữ Exclusive Lock, các Transaction khác không được phép thực hiện các thao tác ghi trên cùng dữ liệu và có thể bị hạn chế đọc tùy theo cơ chế của từng DBMS và Isolation Level.

- **Cấp độ khoá**

+ Khoá cấp cơ sở dữ liệu: Với khoá cấp độ cơ sở dữ liệu, toàn bộ cơ sở dữ liệu bị khoá - điều đó có nghĩa là chỉ một database session có thể áp dụng bất kỳ cập nhật nào cho cơ sở dữ liệu. Loại khoá này không thường được sử dụng, vì nó ngăn tất cả người dùng ngoại trừ một người cập nhật trong cơ sở dữ liệu.

+ Khoá cấp tệp tin: Với khoá cấp tệp tin, toàn bộ một database file bị lock. Do sự đa dạng trong dữ liệu được lưu trong 1 file, cấp độ khoá này không được nhiều người sử dụng.

+ Khoá cấp bảng: Với khoá cấp bảng, toàn bộ bảng bị khoá, cấp khoá này có ích khi thực hiện thay đổi ảnh hưởng đến toàn bộ bảng.

+ Khoá cấp trang hoặc khối

+ Khoá cấp cột: Một số cột của một hàng nhất định bị khoá, ít phổ biến do đòi hỏi nhiều tài nguyên để kích hoạt và giải phóng.

+ Khoá cấp hàng: Áp dụng cho một hàng trong bảng. Đây là cấp khoá phổ biến nhất.

4.4 Partition

- **Khái niệm**

Partition (phân vùng dữ liệu) là kỹ thuật chia một bảng hoặc chỉ mục lớn thành nhiều phần nhỏ gọi là Partition, trong khi về mặt logic chúng vẫn được xem là một bảng duy nhất.

Mỗi Partition chứa một tập con của dữ liệu và được lưu trữ riêng biệt. Khi truy vấn, hệ quản trị cơ sở dữ liệu (DBMS) có thể chỉ truy cập vào các Partition chứa dữ liệu cần thiết thay vì phải quét toàn bộ bảng, từ đó cải thiện hiệu năng truy vấn và quản lý dữ liệu.

- **Cơ chế hoạt động**

Partition hoạt động dựa trên một thuộc tính gọi là **Partition Key**, là cột được sử dụng để xác định bản ghi sẽ được lưu vào Partition nào.

Khi tạo bảng có Partition, người quản trị cần chỉ định:

- Partition Key: cột dùng để phân chia dữ liệu.
- Phương pháp phân vùng: Range, List, Hash hoặc Key Partition.
- Quy tắc phân vùng: quy định giá trị nào sẽ thuộc Partition nào.

Khi một bản ghi mới được thêm vào bảng, DBMS sẽ:

- Đọc giá trị của Partition Key.
- Áp dụng quy tắc phân vùng đã được định nghĩa.
- Xác định Partition phù hợp.
- Lưu bản ghi vào Partition tương ứng.

- **Các loại Partition**

+ Range Partition: Dữ liệu được chia theo các khoảng giá trị của Partition Key.

Ví dụ:

Partition 1: 2020 - 2021

Partition 2: 2021 - 2022

Partition 3: 2022 - 2023

+ List Partition: Dữ liệu được chia theo danh sách các giá trị xác định trước.

Ví dụ:

Partition 1 (North): Hà Nội, Hải Dương.

Partition 2 (Central): Đà Nẵng, Huế.

Partition 3 (South): TP. Hồ Chí Minh, Cần Thơ.

+ Hash Partition: DBMS sử dụng hàm băm để phân phối dữ liệu vào các Partition.

Ví dụ:

CustomerID % 4

0, 4, 8, ... → Partition 0

1, 5, 9, ... → Partition 1

2, 6, 10, ... → Partition 2

3, 7, 11, ... → Partition 3

Hash Partition giúp phân bố dữ liệu đồng đều giữa các Partition. Thường được sử dụng khi không có một dải giá trị rõ ràng để phân chia.

4.5 Sharding

- **Khái niệm**

Sharding là kỹ thuật phân tán dữ liệu bằng cách chia một cơ sở dữ liệu lớn thành nhiều phần nhỏ gọi là Shard, trong đó mỗi Shard được lưu trữ trên một máy chủ (Server) hoặc một cơ sở dữ liệu riêng biệt.

Mỗi Shard chứa một tập con của toàn bộ dữ liệu và có thể hoạt động độc lập. Khi có yêu cầu truy vấn, hệ thống sẽ xác định Shard chứa dữ liệu cần thiết và chuyển truy vấn đến đúng máy chủ, giúp giảm tải cho một máy chủ duy nhất và tăng khả năng mở rộng của hệ thống.

Khác với Partition, Sharding phân tán dữ liệu trên nhiều máy chủ, trong khi người dùng vẫn nhìn thấy dữ liệu như một hệ thống thống nhất.

- **Cơ chế hoạt động**

Sharding hoạt động dựa trên một thuộc tính gọi là Shard Key.

Shard Key là cột được sử dụng để xác định bản ghi sẽ được lưu vào Shard nào.

Khi có dữ liệu mới được thêm vào hệ thống, DBMS hoặc tầng ứng dụng sẽ:

1. Đọc giá trị của Shard Key.
2. Áp dụng quy tắc phân phối dữ liệu.
3. Xác định Shard cần lưu trữ.
4. Gửi dữ liệu đến máy chủ tương ứng.

Khi thực hiện truy vấn, hệ thống cũng sử dụng Shard Key để định tuyến (Routing) yêu cầu đến đúng Shard thay vì tìm kiếm trên toàn bộ hệ thống.

- **Các loại Sharding**

+ Range Sharding: Dữ liệu được phân chia theo khoảng giá trị của Shard Key.

+ Hash Sharding: Hệ thống sử dụng hàm băm (Hash Function) để xác định Shard.

+ Directory Sharding: Hệ thống sử dụng một bảng ánh xạ (Lookup Table) để xác định dữ liệu thuộc Shard nào.

Partition và Sharding đều là kỹ thuật chia nhỏ dữ liệu, nhưng chúng hoạt động ở hai mức khác nhau.

- Partition chia dữ liệu thành nhiều phần trong cùng một cơ sở dữ liệu nhằm tối ưu truy vấn và quản lý dữ liệu.
- Sharding chia dữ liệu thành nhiều phần và phân tán các phần đó lên nhiều máy chủ hoặc nhiều cơ sở dữ liệu, nhằm tăng khả năng mở rộng của toàn hệ thống.

Trong các hệ thống quy mô lớn, hai kỹ thuật này thường được kết hợp. Mỗi Shard lưu trên một máy chủ riêng, còn bên trong mỗi Shard, dữ liệu tiếp tục được Partition theo thời gian hoặc theo một tiêu chí khác để tối ưu hiệu năng truy vấn.

4.6 Isolation

- **Khái niệm**

Isolation (tính cô lập) là một trong bốn thuộc tính của mô hình ACID, đảm bảo rằng các Transaction đang thực hiện đồng thời không gây ảnh hưởng đến nhau. Mỗi Transaction được thực hiện như thể nó là Transaction duy nhất đang truy cập cơ sở dữ liệu.

Mục tiêu của Isolation là đảm bảo kết quả thực thi đồng thời tương đương với việc các Transaction được thực hiện tuần tự, từ đó duy trì tính nhất quán của dữ liệu.

Tuy nhiên, mức độ cô lập càng cao thì khả năng xử lý đồng thời càng giảm do các Transaction phải chờ nhau nhiều hơn. Vì vậy, hầu hết các hệ quản trị cơ sở dữ liệu cung cấp nhiều Isolation Level để cân bằng giữa tính nhất quán và hiệu năng.

- **Cơ chế hoạt động**

Isolation được thực hiện thông qua các cơ chế kiểm soát truy cập đồng thời (Concurrency Control), chủ yếu bao gồm:

+ Locking (Khóa dữ liệu): sử dụng Shared Lock và Exclusive Lock để kiểm soát quyền đọc và ghi dữ liệu.

+ MVCC (Multi-Version Concurrency Control): tạo nhiều phiên bản của cùng một bản ghi, cho phép Transaction đọc dữ liệu mà không bị chặn bởi các Transaction đang ghi.

Tùy từng hệ quản trị cơ sở dữ liệu, Isolation có thể được hiện thực bằng Locking, MVCC hoặc kết hợp cả hai.

- **Các Isolation Level**

1. Read Uncommitted

Read Uncommitted là mức Isolation thấp nhất. Transaction được phép đọc dữ liệu ngay cả khi dữ liệu đó chưa được Transaction khác xác nhận bằng lệnh COMMIT.

Do có thể đọc dữ liệu chưa được xác nhận nên mức Isolation này có hiệu năng cao, tuy nhiên dễ dẫn đến các kết quả không chính xác nếu Transaction cập nhật sau đó bị ROLLBACK.

2. Read Committed

Read Committed chỉ cho phép Transaction đọc những dữ liệu đã được COMMIT.

Nếu một Transaction khác đang cập nhật dữ liệu nhưng chưa COMMIT, Transaction hiện tại sẽ không nhìn thấy sự thay đổi đó.

Đây là mức Isolation mặc định của Oracle và Microsoft SQL Server.

3. Repeatable Read

Repeatable Read đảm bảo rằng nếu một Transaction đọc một bản ghi nhiều lần thì kết quả luôn giống nhau trong suốt Transaction.

Nếu Transaction khác cập nhật bản ghi đó và **COMMIT**, Transaction hiện tại vẫn nhìn thấy giá trị ban đầu.

4. Serializable

Serializable là mức Isolation cao nhất.

Các Transaction được thực hiện như thể chúng diễn ra lần lượt, không có Transaction nào thực sự chạy đồng thời.

Điều này đảm bảo dữ liệu luôn nhất quán nhưng làm giảm đáng kể khả năng xử lý đồng thời của hệ thống.

4.7 Connection Pooling

- **Khái niệm**

Connection Pooling (Nhóm kết nối) là kỹ thuật quản lý kết nối giữa ứng dụng và hệ quản trị cơ sở dữ liệu (DBMS) bằng cách tạo sẵn và duy trì một tập hợp các kết nối (Connection) để nhiều yêu cầu từ ứng dụng có thể tái sử dụng.

Thay vì tạo một kết nối mới mỗi khi ứng dụng cần truy cập cơ sở dữ liệu và đóng kết nối sau khi hoàn thành, Connection Pooling cho phép ứng dụng mượn một kết nối có sẵn trong Pool, sử dụng xong thì trả lại Pool để các yêu cầu khác tiếp tục sử dụng.

Mục tiêu của Connection Pooling là giảm chi phí tạo kết nối, tăng hiệu năng và nâng cao khả năng xử lý đồng thời của hệ thống.

- **Cơ chế hoạt động**

Khi ứng dụng khởi động, một số lượng kết nối đến cơ sở dữ liệu sẽ được tạo trước và lưu trong Connection Pool.

Khi có yêu cầu truy cập cơ sở dữ liệu:

1. Ứng dụng gửi yêu cầu đến Connection Pool.
2. Connection Pool cấp một kết nối đang rảnh.
3. Ứng dụng sử dụng kết nối để thực hiện các câu lệnh SQL.
4. Sau khi hoàn thành, kết nối không bị đóng mà được trả lại Pool.

5. Kết nối sẵn sàng phục vụ các yêu cầu tiếp theo.

5. Nhận xét, lưu ý trong quá trình truy vấn

- Nên tạo index cho các cột thường xuyên sử dụng trong WHERE, JOIN, ORDER BY, GROUP BY.

Do các cột này thường xuyên dùng để tìm kiếm, sắp xếp hay nối bảng nên sử dụng Index sẽ giảm số lượng bản ghi phải quét, từ đó tăng tốc độ truy vấn SELECT.

- Có thể thay điều kiện OR bằng UNION ALL để tận dụng Index.

Khi câu truy vấn chứa nhiều điều kiện **OR**, MySQL có thể không tận dụng được Index và phải quét toàn bộ bảng. Trong một số trường hợp, tách thành nhiều truy vấn và ghép bằng **UNION ALL** sẽ giúp tối ưu hơn.

- Hạn chế sử dụng LIKE '%...'

Không nên đặt ký tự **%** ở đầu chuỗi khi tìm kiếm vì MySQL không thể sử dụng Index, dẫn đến phải quét toàn bộ bảng.

- Sử dụng Full-text Search khi cần tìm kiếm văn bản

Đối với các truy vấn tìm kiếm từ khóa trong văn bản dài hoặc bắt buộc phải tìm theo nhiều vị trí trong chuỗi, nên sử dụng **Full-text Search** thay cho **LIKE '%...%'** để tăng hiệu năng.

- Không nên sử dụng hàm trên cột đã đánh Index (ví dụ YEAR(), MONTH(), UPPER())

Việc sử dụng các hàm trực tiếp trên cột có Index sẽ khiến MySQL không thể dùng Index đó.

- Thiết kế Schema hợp lý

Việc lựa chọn kiểu dữ liệu phù hợp và hạn chế sử dụng NULL giúp giảm dung lượng lưu trữ, tăng hiệu quả xử lý và giảm chi phí truy vấn.

- Chỉ nên chọn cột cần thiết, không nên SELECT *
- Ưu tiên sử dụng INNER JOIN thay vì LEFT JOIN hoặc RIGHT JOIN để giảm chi phí xử lý